

End to End Evaluation of Fabric-X Committer and Orderer

Every comparison in this section is against the results reported in the SIGMOD’26 paper: its Figure 9 for the committer sweeps and its Figure 7b for throughput against transaction size. That is a different deployment from this one, so a difference between the two is a difference between deployments.

Part I — The committer on its own

1 Setup: the committer machines

Nineteen machines, each two sockets of sixteen Intel Xeon Sapphire Rapids cores at 2.10 GHz — 64 hardware threads, 16 MiB of L3 per socket — with 156 GiB of RAM, running three signature verifiers, one coordinator, one sidecar, one load generator, six validator-committers, twelve YugabyteDB tablet servers and three YugabyteDB masters. The six validator-committers and the three masters sit on nine of the twelve database machines, and no machine runs both a validator-committer and a master.

The coordinator tracks dependencies with the simple manager — one map of keys to waiting transactions, owned by a single goroutine — rather than the sharded graph guarded by a mutex. Every machine on both arms runs Red Hat Enterprise Linux 9.8, kernel 5.14. Storage is two 969 GiB XFS volumes per machine, `noatime`, mounted `/data1` and `/data2`. Each database node passes YugabyteDB both volumes as separate data directories (`--fs_data_dirs`, one path per volume) rather than a stripe. The sidecar’s ledger uses `/data1` alone.

Every transaction slot takes a fresh key, so there are no read-write conflicts except in the panel that introduces them deliberately. There a share of transactions carries one back-reference: an existing key in a read-write slot with a nil expected version, which read validation rejects on the same path a double spend takes. That share is the panel’s x axis, and the reference is drawn from the newest million keys as they stood 300,000 transactions earlier. Blocks come from the load generator’s embedded mock ordering service, which cuts and signs them itself and serves them to the sidecar over the Atomic Broadcast API, so no ordering service is in the path.

Transaction signatures satisfy namespace 0’s endorsement policy, a threshold rule over the endorsing user’s signing certificate. The load generator creates the namespace itself on this arm, registering the policy over a key pair it holds, so the policy and the key it signs with agree by construction and the scheme is a free choice. These figures sign with ECDSA P-256, as Part II’s do.

A measured point is the highest offered rate that arrived in full, committed within 2%, kept p99 under 1s and held a flat in-flight count, confirmed by a 300s hold on a fresh deployment.

2 Why this setup

The shape is not the SIGMOD’26 paper’s, and neither is the hardware. Its committer deployment is nine validator-committers on nine database nodes, and both of its arms ran on AWS: a `c6id.8xlarge` for each validator-committer with its database node (32 vCPUs of Ice Lake at 3.5 GHz, 64 GiB), a `c6id.16xlarge` each for the coordinator, the sidecar and the load generator, and a `c5a.8xlarge` for each verifier (32 vCPUs of EPYC at 3.3 GHz). Storage on both arms is EBS General Purpose, rated at 3,000 IOPS and 125 MiB s^{-1} — the `c6id` local NVMe was not used. This deployment is wider and slower per core — 64 hardware threads at 2.10 GHz a machine — with more memory, and with local volumes that Table 2 measures at two orders of magnitude more random write IOPS than that EBS rating. What follows therefore compares deployments, not matched hardware.

Both volumes, not a stripe, so YugabyteDB sees the disk boundary and places data across it itself.

A conflict-free workload measures the pipeline’s ceiling rather than its conflict behaviour, at the cost of a state database and a ledger that grow with every transaction.

The reference gap is derived, not copied. A back-reference is only a double spend if the key it names is already committed; a reference to a key still in flight is a dependency the coordinator has to order, which is a different measurement wearing the same label. How far back a reference must reach to be safe is a property of the deployment, not of the workload: in-flight keys are the rate times the latency times the fresh keys a transaction introduces, so 304,000 to 402,000 keys here at 518,000 tps and a 345 ms to 457 ms pipeline. A gap of 300,000 transactions is 510,000 keys and clears all of it. The SIGMOD’26 gap of 1,000 transactions clears its own 85 ms pipeline and would leave two fifths of these references inside this one, which is why the parameter is derived here rather than copied. Erring long costs nothing — an older key is still a double spend — and erring short would have cost the result: the database raises `unique_violation` only for a key it has already committed, so a gap inside the in-flight window routes most references into dependency ordering and never reaches the insert path that carries the cost reported for the double-spend panel. Conflicts are then introduced deliberately as their own variable.

A flat in-flight count is required because throughput alone cannot tell a rate the pipeline sustains from one it is absorbing into a growing backlog: both commit everything offered for as long as the queue has room to grow. What separates them is latency, since a transaction waits in that queue before anything processes it, and what happens next — the queue is bounded, so a rate above capacity collapses once it fills.

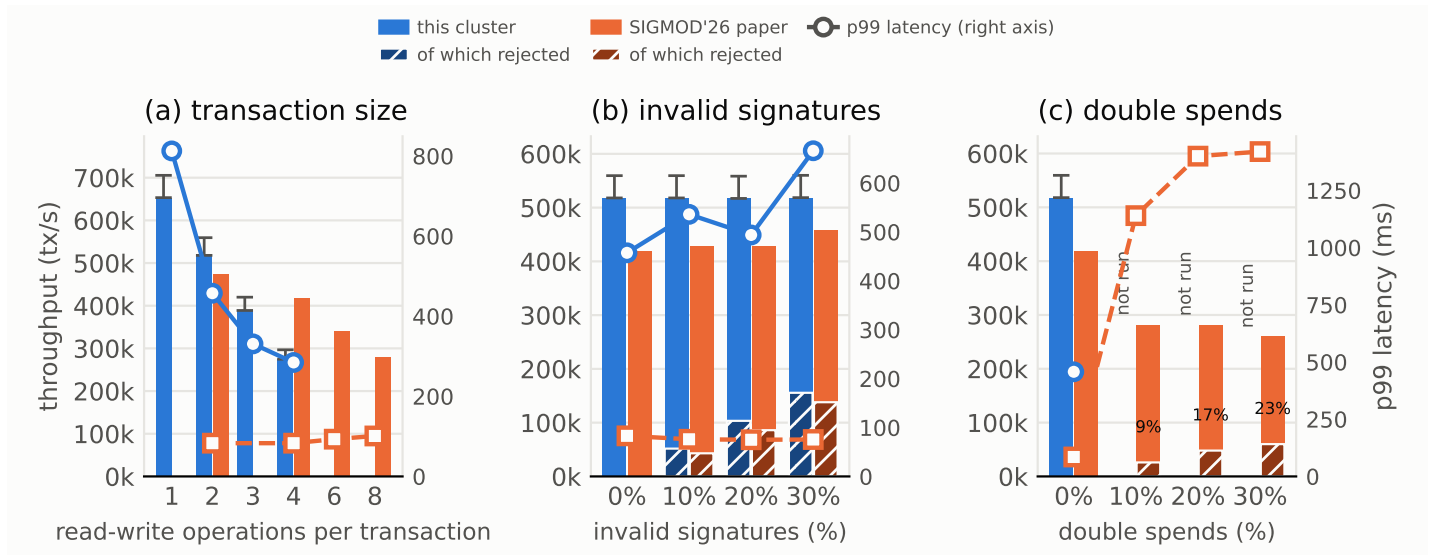


Figure 1: Committer throughput against (a) transaction size, (b) invalid-signature share and (c) double-spend share, beside the SIGMOD’26 paper’s Figure 9. Bars are throughput on the left axis; the marks above them are the 99th-percentile latency of the same measurement on the right axis, in that series’ own colour. Throughput counts transactions finished, committed plus rejected, and the hatched part of a bar is the rejected share — labelled with the share it measured only in (c), where what the generator produced differs from what was asked for; elsewhere it is the x value. Whiskers are the search’s 8% step, one-sided. *Not run* marks a condition never attempted at that configuration.

3 Throughput and tail latency

Throughput falls with transaction size (Fig. 1a), 653,273 tps at one read–write operation to 274,727 at four. Comparing that with the SIGMOD’26 figure needs the axes aligned: its x axis counts inputs and outputs, and one in with one out is a *two* read–write total, so its N/N sits at our $2N$. Only two of our four sizes therefore have a published counterpart. At two read–writes this cluster holds 518,000 tps against the SIGMOD’26 474,000, 9% ahead; at four it holds 274,727 against 419,000, 34% behind. Over those same two sizes our throughput falls 47% where the SIGMOD’26 sweep falls 12%, so whatever this pipeline pays per key, it pays much more steeply. Its six- and eight-key points were never measured here, and our one- and three-key points have nothing to compare against.

The mechanism behind that slope is not the SIGMOD’26 one, which is lock contention in the coordinator’s dependency graph. There is no such lock here: the simple manager has one owning goroutine where the sharded graph has a mutex, and its two lock-wait histograms read zero in every one of 2,326 thirty-second samples across these runs. What the graph costs instead is per key: a stage benchmark of the manager puts it at about 420ns a key, flat from one key per transaction to eight, which is a fall of the shape this panel shows. Running the sharded graph on this deployment instead does register on those histograms, so the difference is the manager and not the instrument.

Read as a key rate rather than a transaction rate, the difference between the two sweeps has a shape. Multiplying each point by its keys per transaction gives 653,273, 1,036,000, 1,166,727 and 1,098,908 keys a second at one to four keys: it climbs to about 1,170,000 and stops. Above one key this pipeline is therefore bounded per key, and its

transaction rate falls because each transaction spends more keys out of a fixed key budget. The SIGMOD’26 sweep does not reach a boundary over its range — 948,000 keys a second at two keys, then 1,676,000, 2,040,000 and 2,240,000 at four, six and eight. At two keys our key rate is the higher of the two; by four theirs is 53% above ours and still rising.

Where that key budget goes is a hypothesis rather than a result, and the storage path is not the answer. Every key costs a version lookup and a write in the state database, but the published deployment’s volumes are rated at 3,000 IOPS and 125 MiB s^{-1} against the 102,000 random-write IOPS and $1,058 \text{ MiB s}^{-1}$ Table 2 measures here, so whatever the key budget is spent on, it is not disk this deployment lacks. Neither ceiling is disk-bound at these rates in any case: nine volumes at that rating supply 27,000 IOPS against the 1,676,000 keys a second the published sweep reaches at four keys, so both deployments must serve reads from memory and batch their writes.

That leaves the compute side, where the two differ in opposite directions. Each published validator–committer runs on 32 vCPUs at 3.5 GHz, 67% more clock per core than here; this deployment answers with more cores and more memory per machine, and with six validator–committers against nine. So the published pipeline is the faster one per key and this one the wider one, which is consistent with a per-key cost that is partly serialized — and that is as far as a ladder on one deployment can place it. Settling it needs the two tier sizes measured at a matched clock, which nothing here provides.

Invalid signatures are close to free (Fig. 1b): 518,000 tps with none, then 518,000, 517,091 and 518,545 at 10%, 20% and 30% — all four flat within 0.3%, far inside this cluster’s 8.8% repeatability, so rejection costs nothing measurable in throughput. It is not the SIGMOD’26 10% *gain* either.

Double spends are not free here, and what they cost is not conflict resolution. At the tablet layout this deployment uses, a 5% double-spend share commits 19,929 to 20,617 tps — flat within 3.4% — at every offered rate from 25,000 to 500,000, a twentyfold range, read from the validator–committers’ own counters. It cannot be driven past that: the generator sent 21,000 to 22,000 tps at every rung, held there by the sidecar’s window, so beyond the lowest rung the offered rate never reaches the pipeline. What a conflicting workload costs, then, is a ceiling near 20,300 tps, a factor of twenty-five below the 518,000 the same pipeline sustains with none.¹

The cost is work done in the database rather than time spent queueing, and three measurements place it there. Under conflicts the busiest machine spends *twenty-two times* the CPU per transaction that it spends without them, and no amount of queueing makes a transaction cost more CPU.² The insert step of a commit takes 1.72 s a call at 1.91 calls a commit, the second call being a retry. And the same workload costs a fraction of that at a smaller tablet layout, which the sweep below quantifies — so the penalty belongs to the layout rather than to conflicts.

The mechanism is visible in the state database’s own procedure. A batch of new keys is inserted blind and returns on success, performing no lookup at all. One key that already exists raises a uniqueness violation, and the handler then looks up *every* key the batch carries to report which ones collided. The batch is then rolled back and retried, which is the second insert call. So a conflict-free workload never performs that lookup, and one conflicting key makes a whole batch perform it — together with a rollback of a distributed transaction that has to reach every tablet the batch touched.

The failure path is what costs. Backing the clean attempt out of the insert histogram puts a failing attempt between 1.2 and 2.6 s, against the 23 ms a whole clean commit takes. It is not an effect of the table growing: across an eightfold growth at fixed batch width the insert moves by 0.8%.

The commit itself, not the queue in front of it, is what exceeds the bound. At 96 tablets an offered 15,659 tps is retired in full with a flat in-flight count, and there the insert takes 1.76 s at 1.90 attempts — 3.35 s inside one commit. The same quantity falls to 2.29 s where the pipeline is nearly idle, so it is a cost that varies with load rather than a constant, but neither figure fits inside a second. Both are lower bounds on the commit, since the measurement brackets the insert alone and leaves the distributed transaction’s rollback and the retry loop outside it.

The layout, however, is decisive, and a sweep of it at one fixed workload — 5% double spends throughout — is the whole of the effect:

¹The load generator’s own rate for the same rungs scatters from 18,000 to 25,273, a spread of a third, because it measures status arrivals in a sixty-second window while several million transactions are queued ahead of them. Its mean, 20,610, agrees with the committers’ counters; its variance is the queue’s, not the pipeline’s. Figures here are the committers’.

²Busiest-host CPU per cent times its 64 threads, divided by transactions finished a second. That attributes a whole machine to one workload, so it is a comparison and not a per-transaction cost: the ratio is the claim, and the absolute figures behind it are 2,133 μ s against 99 μ s.

tablets	committed	p99	insert	busiest CPU
8	172,260 [†]	239 ms	—	22 %
12	95,123	190 ms	13.6 ms	11 %
88	27,400	—	1.19 s	71 %
96	20,800	—	1.76 s	34 %
120	20,300	—	1.69 s	71 %

Twelve tablets is the deployment’s own default, and it is the only layout that sustains the bound. Its figure is the top of an ascending ladder — 15,000, 30,000, 60,000 and 100,000 tps offered, each held 300 s on one deployment, each arriving in full at a flat in-flight count — and every rung met the bound, so the pipeline sustains *at least* 95,123 tps here: the ladder ran out of rungs rather than the pipeline out of capacity. p99 is 190 ms across the top three rungs and the busiest machine reaches only 11%. That is 4.7 times what the 120-way split commits under the same workload, and the difference between meeting the bound and not.

The insert column is the whole of the effect. At twelve tablets it costs 13.6 ms to 15.2 ms, and it does not move with load: the offered rate rises 6.7-fold across that ladder and the insert *falls*. Above eighty tablets the same step costs more than a second, and there it does move with load — at 96 tablets it runs from 1.62 s to 2.03 s as the offered rate doubles. Since a commit makes 1.90 attempts on this workload, fitting inside a second needs the insert under 0.53 s, which the larger splits miss by a factor of three and the smaller ones by none.

The eight-tablet row (†) is a 90 s probe rather than a ladder; held 300 s at the same offered rate it delivered 125,000 tps at a 25 s mean, so its rate is unconfirmed, and the twelve-tablet ladder passing 95,123 at 11% CPU makes its 172,260 plausible rather than anomalous. The three larger rows are retirement rates under saturation, which a pipeline holds whatever its queue depth; none qualified at any rate measured, and at 96 tablets those rates reach well below capacity — five rungs from 11,313 to 21,675 tps offered, each retiring what it was offered at a flat in-flight count, gave means from 5.6 s to 8.6 s, the lowest at little more than half of capacity and still missing the bound more than fivefold. At 120 tablets no rate below 25,000 has been attempted.

The trade runs the other way on a workload that never violates: there the 120-way split is worth at least 2.4 times as much, 518,000 tps confirmed over 300 s against the 213,091 a 90 s probe reached at twelve tablets and did not hold. So the split is a workload-dependent choice rather than a misconfiguration — it buys write concurrency where nothing collides and costs a factor of 4.7 and the latency bound where things do.

So the conflict figure is a property of the deployment’s storage layout rather than of its conflict handling. This is why the figure is reported here rather than beside the published double-spend sweep — the published deployment’s numbers were taken at a layout this one does not share.

4 Latency against throughput

Both curves carry two read–write operations over 32 B keys, 262 B serialized, and differ only in the number of transactions the generator puts in a block.

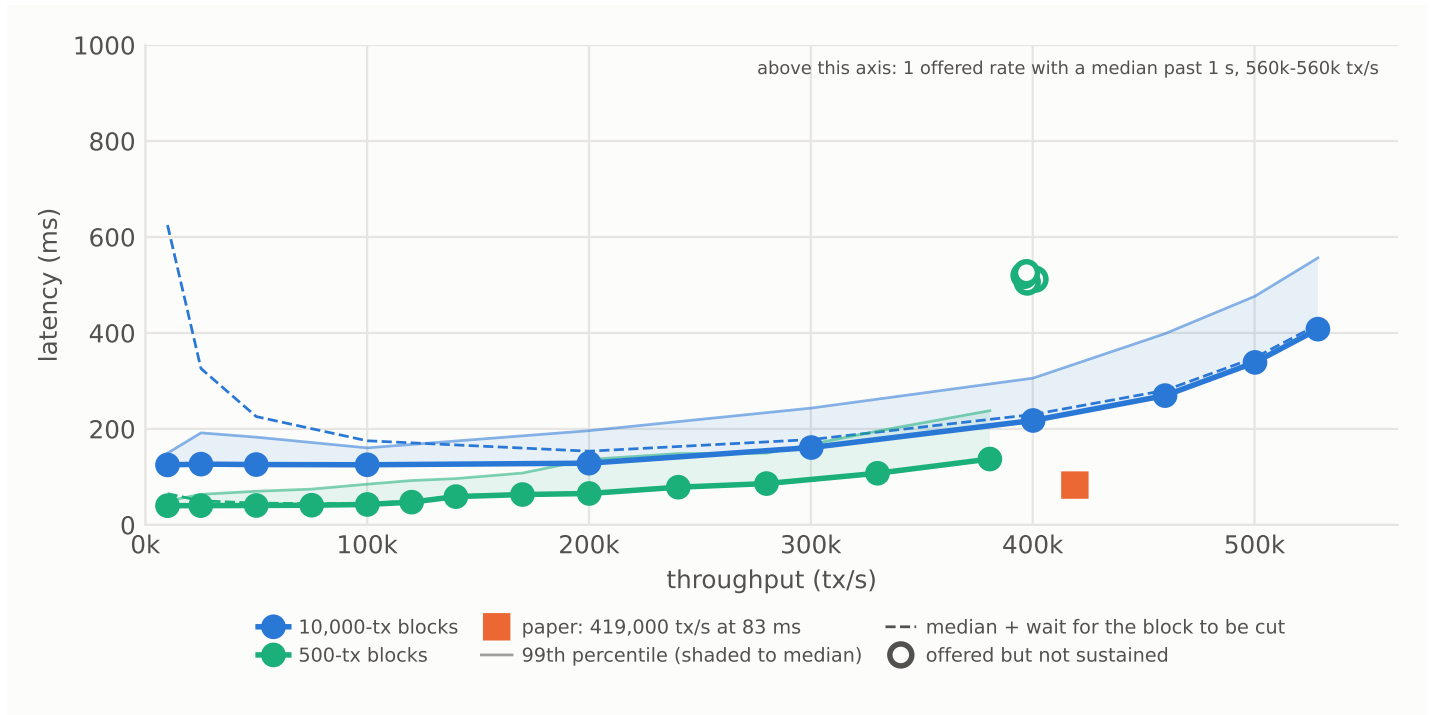


Figure 2: Latency against throughput for the committer alone, at two block sizes and a fixed workload of two read–write operations. Thick line: the median; the thin line above it the 99th percentile, shaded between; dashed: the median plus the mean wait for a block to be cut, which the measurement excludes. The latency axis runs to one second, the bound every reported point meets: it is floored so the operating region stays legible and capped so a rung that took seconds cannot flatten the rest. Offered rates whose median is past it are summarised in a corner rather than drawn, and a hollow marker appears only where one falls inside the axis.

At 10,000-transaction blocks the median has a floor near 125ms that does not fall with the rate, since nothing in a block moves until the block is cut (Fig. 2). At 500 transactions a block the floor is 40 ms, and 379,764 tps holds at 136ms median and 235ms p99 — about the median large blocks give at 10,000 tps, at thirty-eight times the rate.

That ladder stops where the load generator stops, not where the committer does, and the constraint is the interface between them rather than any cost inside either. What bounds the transactions the generator can have outstanding is a buffer counted in *blocks*, so at 500 transactions a block it holds twenty times fewer transactions than at 10,000 for the same rate. Deepening it twentyfold, to match in transactions, moves the small-block ceiling from 396,000 to 400,000 to 430,145 tps sustained at a 209ms median, where the generator plateaus at 21% of its cores. What the committer itself sustains at this block size is therefore still not resolved: every rung measured so far is a property of the instrument.

Part II — End to end, with a real ordering service

The second arm puts a real Arma ordering service in the path. The committer tier is **unchanged from Part I** — the same nineteen machines, the same components, the same tuning — so a difference between the two arms is attributable to ordering.

5 Setup: the ordering machines

Twenty further machines of the same processor, each two sockets of eight cores — 32 hardware threads, 16 MiB of L3 per socket — with 78 GiB of RAM, holding four parties of a router, a consenter, an assembler and one batcher per shard. Routers, consenter and assemblers take a machine each, twelve in all; the batchers share the remaining eight. Two configurations are compared:

- **Four shards.** Sixteen batchers, two to a machine, each with a volume to itself (`/data1` and `/data2`).
- **Eight shards.** Thirty-two batchers, four to a machine, two sharing each volume.

Storage is two 485 GiB XFS volumes per machine, mounted as on the committer machines. Routers, consenter and assemblers write to `/data1`; the batchers are distributed across both volumes as above.

A batcher cuts batches and the assemblers turn each one into a block, so on this arm the block-size knob is the shared configuration's `Batching.BatchSize.MaxMessageCount` rather than a load-generator setting, and `BatchCreationTimeout` is 500 ms. The consensus decision interval, `RequestBatchMaxInterval`, is 50 ms, reduced from the default 500 ms.

6 Why this setup

Two batchers to a machine is measured rather than assumed. At two shards the service capped near 158,000 tps

per shard whatever the batch size or decision interval, while batcher processes sat at 15 % to 20 % of a 32-core machine — so the constraint is per shard and the spare capacity for a second was already on the box.

Both shard counts are measured because that reading predicts more shards buy throughput, and the two configurations separate the two things a shard needs: a core budget and a disk. Four shards give each co-located batcher its own volume; eight give twice the shards but make two share one.

These machines are half a committer machine, and half is the whole of the difference. They are the same processor at the same clock, so a comparison between the arms is one of width rather than of generation. What differs is half the threads, half the memory and, per Table 2, half the disk bandwidth; what favours the ordering machines is twice the L3 per core. So a ceiling measured on this arm belongs to these machines before it belongs to ordering, while a per-core one belongs to neither class in particular.

A 50 ms decision interval because the default 500 ms was the largest single term in end-to-end latency and a ceiling on throughput as well. A consenter sees batch attestations rather than transactions — some 28 a second across two shards at 280,000 tps — so `RequestBatchMaxCount` of 100 would take seconds to fill and the interval is what cuts every decision. Throughput follows the interval for the same reason: at the ceiling each decision carries about the same number of transactions, so more decisions a second is more transactions a second.

7 Latency against throughput, by shard count and storage

Both ladders here carry the workload of Fig. 2 — two read-write operations over 32 B keys, 262 B serialized, every slot taking a fresh key — so the two arms are comparable at matched rates and differ only in whether an ordering service is in the path.

End to end, ordering included, this deployment sustains 499,091 tps at 658 ms p99, with every offered rate up to that arriving in full and no aborted transactions at any rung. Two comparisons place it. The SIGMOD’26 paper has no end-to-end figure, but its ordering service alone reaches 430,000 tps at four parties and four shards — so the complete pipeline here exceeds the published figure for one part of it. That is the one comparison in this section between comparable machines: those experiments ran on 32-vCPU VMs with 64 GiB of memory, against the 32 threads and 78 GiB here, and on EBS volumes rated below what Table 2 measures on these. And the same committer at the same workload without an ordering service holds 518,000 tps, reaching 531,455 on the ladder of Fig. 2, so putting real ordering in the path costs about 6 % of the committer’s own ceiling — inside this cluster’s repeatability rather than a separate order of magnitude.

Both configurations sustain the same throughput and hit the same knee: 499,091 tps at four shards against 500,000 at eight, 0.2 % apart, and neither sustains 550,000 (Fig. 3). Four shards is the better configuration on every other axis — p99 lower at every matched rate, by 44 ms to 248 ms (658 ms

against 754 ms at the knee), fewer processes, and a volume per batcher rather than two sharing one.

Doubling the shards buys nothing because ordering is not what limits this arm. Below 400,000 tps the busiest machine is a router; at the top of both ladders it is a validator–committer at 76 % to 82 %. Extra shards add capacity to the tier that is not binding.

The knee itself cannot be attributed to a single tier. At 500,000 tps no machine anywhere exceeds 82 %: the busiest validator–committer is at 81 % and *the load generator is at 77 %*, four points behind it. So the ceiling is neither a CPU wall on any tier nor separable by this experiment — the two candidates are within each other’s margin, and a ladder on one deployment cannot tell them apart. Naming the busiest host names a correlate, not a cause.

Extra shards also cost latency, for a reason the block size makes plain. At a given total rate each of eight shards sees half the per-shard rate that four do, so batches take twice as long to reach 10,000 transactions and the 500 ms `BatchCreationTimeout` cuts them early over twice as wide a range of rates. The eight-shard curve shows exactly that shape: p99 rises to 793 ms at 150,000 tps, then falls back as the rate grows enough to fill a batch before the timer fires — 687, 614 and 591 ms at 200,000, 250,000 and 300,000 tps.

8 Latency against throughput, by batch size

Batch size is the latency–throughput knob here, and it trades one for the other almost exactly (Fig. 4), at the same workload again. Cutting the batch from 10,000 transactions to 500 removes most of the latency at every rate the two share — p99 197 ms against 693 ms at 100,000 tps, 286 ms against 460 ms at 250,000, and mean latency between a quarter and a half of what large batches give. That is the predicted effect and its direction confirms the mechanism: a batch a twentieth the size fills twenty times sooner, so the 500 ms `BatchCreationTimeout` stops being what cuts it.

What it costs is the ceiling, and the reason is exact. The block rate tracks the offered rate precisely — 200 blocks a second at 100,000 tps, 400 at 200,000, 500 at 250,000 — and between 500 and 600 blocks a second the arm stops working. 250,000 tps is therefore not a transaction limit at all: it is 500×500 , a *block-rate* ceiling near 500 a second. At 10,000 transactions a batch the same block rate would carry five million transactions a second, which is why that configuration is limited by transactions instead and reaches 499,091.

So the two configurations are bounded by different things, and the choice is a real trade rather than a tuning win: small batches halve the latency and halve the ceiling.

9 Throughput against transaction size

Throughput falls with transaction size, but far more slowly than in the SIGMOD’26 sweep, and the byte rate is what

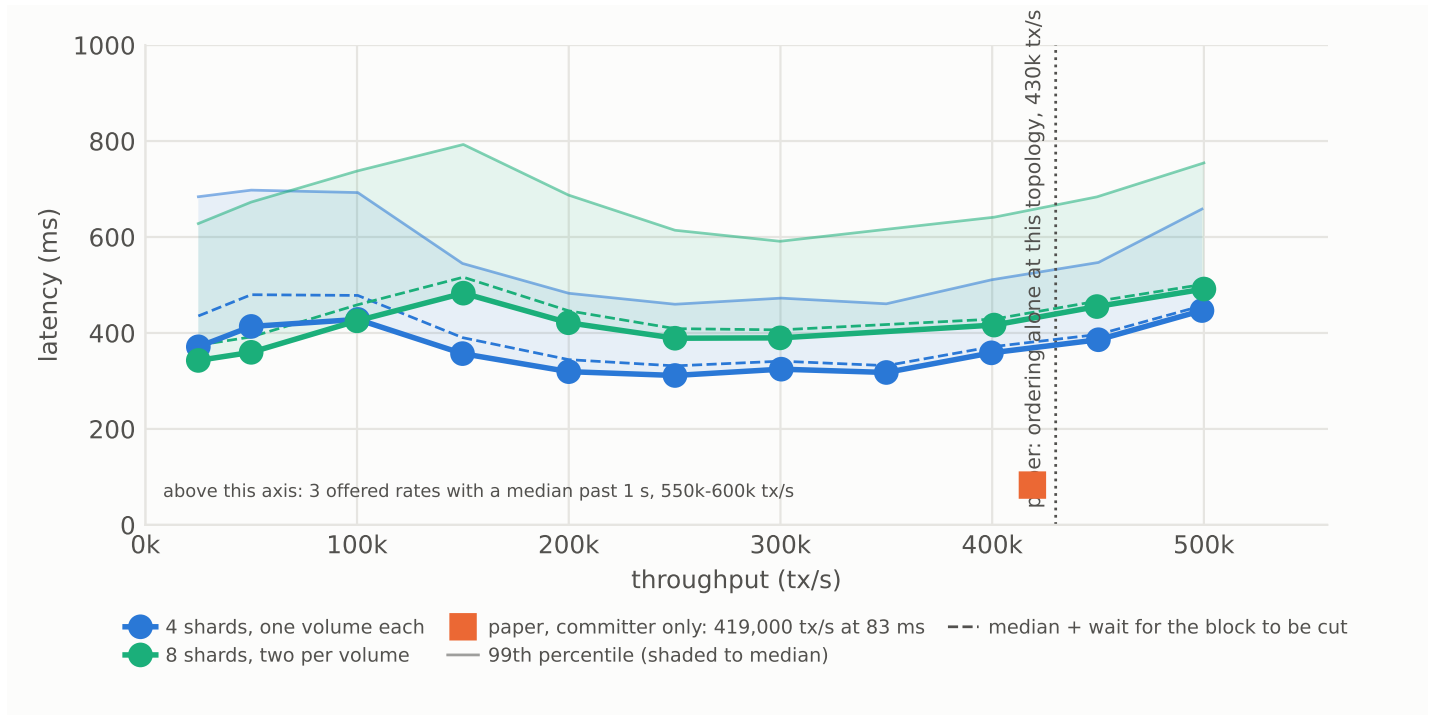


Figure 3: Latency against throughput end to end, with a real ordering service in the path, by shard count and batcher storage layout. Marks as Fig. 2. The dotted vertical line is the SIGMOD'26 ordering-alone figure for this topology.

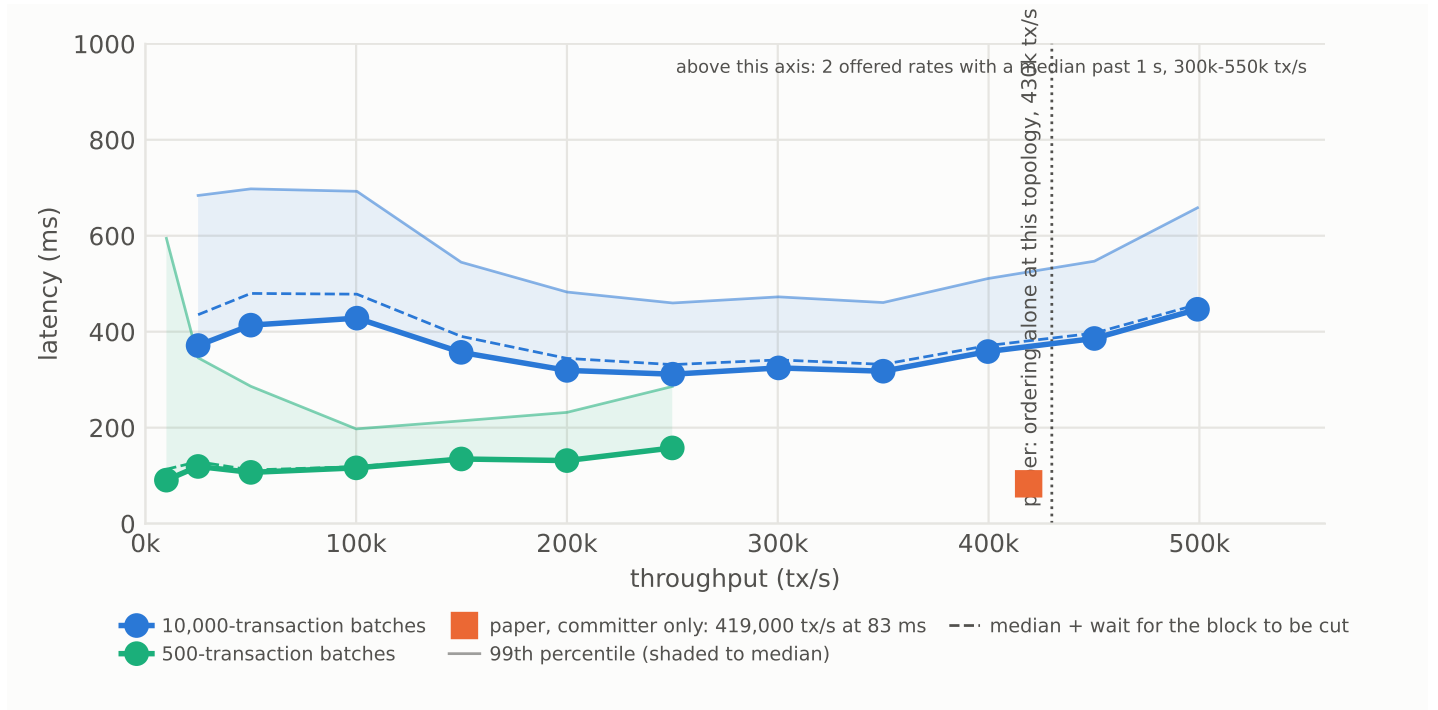


Figure 4: Latency against throughput end to end, by batch size, on four shards with a volume per batcher. Marks as Fig. 2.

shows why (Fig. 5). The sweep keeps the same two read-write operations and varies only the value each one carries, so the serialized envelope is 202 bytes plus twice that value. That measurement is bandwidth-bound past 300B: each of its points sits within 10% of 120 MB s^{-1} , so its rate is simply bytes per second divided by size — and the figure is the cap its own storage carried, since those experiments ran on AWS VMs on EBS General Purpose volumes rated at 125 MiB s^{-1} . This deployment is not bandwidth-bound anywhere in the

range. Its byte rate *rises* with size, from 122 MB s^{-1} at 300 B to 410 MB s^{-1} at 4 KiB, because what it pays per transaction dominates what it pays per byte: halving the transaction count at twice the size is a net gain. Throughput therefore falls about $0.76\times$ per doubling rather than the $0.5\times$ a bandwidth-bound service gives.

The tabulated 512 B figure is nominally above the 300 B one, which is not a real inversion: the 300 B point's two holds span 408,000 to 443,800 tps and 512 B sits inside that, so the

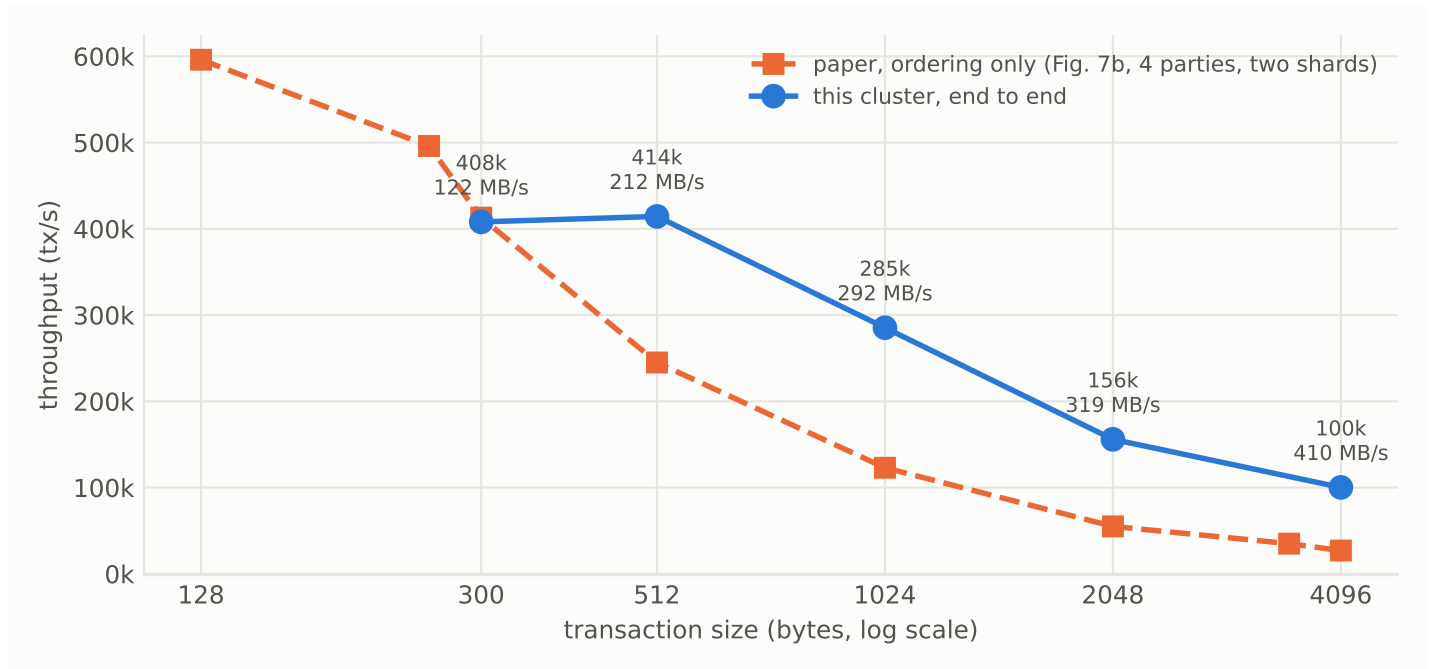


Figure 5: Throughput against transaction size end to end, beside the SIGMOD’26 paper’s Figure 7b. Each point is annotated with the byte rate it implies.

sweep is flat between those sizes rather than rising. It is the kind of difference the repeatability figure in Table 1 exists to rule out.

Where that stops is measurable, and it is the assembler’s disk. At 4 KiB the arm delivers 113,000 to 123,000 tps whether it is offered 118,000 or 163,200, so throughput plateaus near 120,000 tps — 492 MB s^{-1} of envelope bytes — with no machine anywhere above 54% CPU, so it is not a processing limit. The highest rate that also meets the latency bound, which is what Table 1 reports, is lower still at 100,000 tps. The assemblers at that point are writing 527 MB s^{-1} to 546 MB s^{-1} to their data volume, against the 529 MiB/s sequential write Table 2 measures on these machines: the device is saturated. Each of the four assemblers writes the *complete* block stream, where a batcher writes only its own shard’s share, so the assembler is where a byte rate first meets a disk. At 2 KiB the same figure is 319 MB s^{-1} , three fifths of that ceiling, which is why that point holds cleanly and this one does not.

So the sweep has two regimes rather than one. Below 2 KiB the arm is limited per transaction and the byte rate climbs; at 4 KiB it is limited per byte by one component’s disk, and the SIGMOD’26 bandwidth-bound model finally applies here too, at about four times the bandwidth.

The consequence for the comparison is that only the direction is shared. Both lines fall, and this one is above the SIGMOD’26 line at every size they have in common — but it is a whole pipeline against one of its parts, at four shards against two, so the gap is not a speedup and the shapes are not measuring the same limit.

Table 1: The size sweep end to end, with the evidence class of each point, since they do not all meet the same bar. A hold is tabulated in preference to a higher probe wherever both exist. At 1 KiB and above a hold does not fit in one deployment: each assembler writes the complete block stream, so a 485 GB volume fills in under twenty minutes at these byte rates, less than a rate search followed by a 300 s hold. The 300 B point is the lower of two holds 8.8% apart, so all figures are given to three significant figures.

Size	Throughput (tx/s)	Byte rate	Evidence
300 B	408,000	122 MB/s	two holds, lower one
512 B	414,000	212 MB/s	confirmed hold
1 KiB	285,000	292 MB/s	probe; volume filled
2 KiB	156,000	319 MB/s	confirmed hold
4 KiB	100,000	410 MB/s	probe; disk-bound

10 Storage characterisation

Two properties of these volumes bound the results above, and both had to be measured: the volumes are *virtio* devices reporting neither a model nor a meaningful rotational flag, so the backing media cannot be identified from inside the guest. Table 2 gives *fio* 3.35 in the four patterns this deployment produces.

First, ordering machines have half the disk bandwidth as well as half the cores, at almost exactly a factor of two on every bandwidth row, at round figures repeating to four significant figures across machines and volumes — these are provisioned caps, not device characteristics, so a ceiling on that arm must be read against the cap before it is attributed to ordering. The synchronous pattern is the exception, where the ordering volumes are the faster of the two ($297 \mu\text{s}$ against $395 \mu\text{s}$ at the median). Second, durability is the expensive direction: making one 4 KiB record durable with an `O_DSYNC` write costs about $400 \mu\text{s}$, capping any per-record design near 2,400 operations a second, three orders of magnitude below

Table 2: `fio 3.35, libaio, direct=1`, 20 s per pattern, run on a volume the deployment was not using at the time. Committer figures are the range across a database node and the sidecar. A synchronous engine caps `fio`’s queue depth at 1 while still reporting the depth asked for, so the deep rows use `libaio`.

Pattern	Depth	Committer	Ordering
1 MiB seq. write	4	1,058 MiB/s	529 MiB/s
4 KiB O_DSYNC write	1	2,400 IOPS	3,249 IOPS
8 KiB rand. read	32	126,000 IOPS	67,689 IOPS
8 KiB rand. write	32	102,000 IOPS	51,165 IOPS

the rates measured here. The ledger survives that only by batching — it appends a whole block and syncs every hundredth. Syncing alone is cheap by comparison, an `fdatasync` after a buffered write returning in about 0.08 ms: it is the write, not the barrier, that costs.

11 Threats to validity

The load generator is the instrument, and in one figure it is the limit: its block preparation caps the small-block curve in Fig. 2. Elsewhere it is close to being one. At the end-to-end knee it runs at 77 % of its 64 cores against the busiest validator–committer’s 81 %, and on the 300 B size point at 69 % against 75 % — close enough that neither is clearly the constraint. Every figure near 500,000 tps should therefore be read as a bound on the pair, generator and pipeline together, not on the pipeline alone. Separating them needs a second generator, and there is no spare machine for one without taking it from the committer. A knee is resolved to the search’s 8 % step, so it is a lower bound, drawn one-sided in Fig. 1. The baseline drifts 10 % between days, so only same-day comparisons are used, and within a day repeatability is no better: one configuration measured twice gave holds 8.8 % apart. A difference under about a tenth is therefore not a result here, and no claim in this section rests on one.

A ladder climbs on a single deployment, so rate and table fill rise together along it. Repeating the rates around the knee on fresh deployments moves the boundary by less than the spread above, so the knee is a property of the rate; the tail near it does grow with fill. Every rung records the fill it ran against.